

Отчет по блоку 4

Алгоритмы и структуры данных

Содержание

1	Задача М	2
2	Задача N	5
3	Задача O	7
4	Задача P	9

1 Задача М

Текст задания

М. Цивилизация

✓ Полное решение

Ограничение времени	1.5 секунд
Ограничение памяти	256 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Карта мира в компьютерной игре «Цивилизация» версии 1 представляет собой прямоугольник, разбитый на квадратики. Каждый квадратик может иметь один из нескольких возможных рельефов, для простоты ограничимся тремя видами рельефов — поле, лес и вода. Поселенец перемещается по карте, при этом на перемещение в клетку, занятую полем, необходима одна единица времени, на перемещение в лес — две единицы времени, а перемещаться в клетку с водой нельзя.

У вас есть один поселенец, вы определили место, где нужно построить город, чтобы как можно скорее завладеть всем миром. Найдите маршрут переселенца, приводящий его в место строительства города, требующий минимального времени. На каждом ходе переселенец может перемещаться в клетку, имеющую общую сторону с той клеткой, где он сейчас находится.

Формат ввода

Во входном файле записаны два натуральных числа N и M , не превосходящих 1000 — размеры карты мира (N — число строк в карте, M — число столбцов). Затем заданы координаты начального положения поселенца x и y , где x — номер строки, y — номер столбца на карте ($1 \leq x \leq N$, $1 \leq y \leq M$), строки нумеруются сверху вниз, столбцы — слева направо. Затем аналогично задаются координаты клетки, куда необходимо привести поселенца.

Далее идет описание карты мира в виде N строк, каждая из которых содержит M символов. Каждый символ может быть либо «.» (точка), обозначающим поле, либо «W», обозначающим лес, либо «#», обозначающим воду. Гарантируется, что начальная и конечная клетки пути переселенца не являются водой.

Формат вывода

В первой строке выходного файла выведите количество единиц времени, необходимое для перемещения поселенца (перемещение в клетку с полем занимает 1 единицу времени, перемещение в клетку с лесом — 2 единицы времени). Во второй строке выходного файла выведите последовательность символов, задающих маршрут переселенца. Каждый символ должен быть одним из четырех следующих: «N» (движение вверх), «E» (движение вправо), «S» (движение вниз), «W» (движение влево). Если таких маршрутов несколько, выведите любой из них.

Если дойти из начальной клетки в конечную невозможно, выведите число — 1.

Пример

Ввод

```
4 8 1 1 4 8
...WWW
.#####.
.#..W...
...WWW.
```

Вывод

```
13
SSSEENEEEEES
```

Пояснение к алгоритму

Карта представляется как граф, где каждая свободная клетка является вершиной. Из клетки можно перейти в соседние по стороне клетки, если там нет стены. Стоимость перехода зависит от типа клетки: обычная клетка стоит 1, а клетка с водой стоит 2. Для поиска минимальной стоимости пути используется алгоритм Дейкстры, потому что

все веса переходов положительные. Чтобы восстановить маршрут, для каждой клетки запоминается предыдущая клетка в кратчайшем пути.

Сложность по времени

$O(nm \log(nm))$, где n и m - размеры поля.

Сложность по памяти

$O(nm)$.

Код

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m, r, c, x, y;
6     cin >> n >> m >> r >> c >> x >> y;
7     r--, c--, x--, y--;
8
9     vector<string> g(n);
10    for (auto& s : g)
11        cin >> s;
12
13    const long long I = 1e18;
14    vector d(n, vector(m, I));
15    vector p(n, vector<pair<int, int>>(m, {-1, -1}));
16
17    priority_queue<tuple<long long, int, int>, vector<tuple<long long, int,
18        int>>, greater<>> q;
19
20    d[r][c] = 0;
21    q.push({0, r, c});
22
23    int dr[] = {-1, 0, 1, 0};
24    int dc[] = {0, 1, 0, -1};
25
26    while (!q.empty()) {
27        auto [w, i, j] = q.top();
28        q.pop();
29        if (w > d[i][j])
30            continue;
31        for (int k = 0; k < 4; k++) {
32            int ni = i + dr[k], nj = j + dc[k];
33            if (ni < 0 || ni >= n || nj < 0 || nj >= m || g[ni][nj] == '#')
34                continue;
35            long long nw = w + (g[ni][nj] == 'W' ? 2 : 1);
36            if (nw < d[ni][nj]) {
37                d[ni][nj] = nw;
38                p[ni][nj] = {i, j};
39                q.push({nw, ni, nj});
40            }
41        }
42    }
43
44    if (d[x][y] == I) {
45        cout << -1;
46        return 0;
47    }
48
49    cout << d[x][y] << '\n';
```

```

50  string s;
51  for (int i = x, j = y; i != r || j != c;) {
52      auto [pi, pj] = p[i][j];
53      if (pi == i - 1)
54          s += 'S';
55      else if (pi == i + 1)
56          s += 'N';
57      else if (pj == j - 1)
58          s += 'E';
59      else
60          s += 'W';
61      i = pi;
62      j = pj;
63  }
64
65  reverse(s.begin(), s.end());
66  cout << s;
67 }

```

2 Задача N

Текст задания

N. Свинки-копилки

✓ Полное решение

Ограничение времени	1 секунда
Ограничение памяти	256 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

У Васи есть n свинок-копилков, свинки занумерованы числами от 1 до n . Каждая копилка может быть открыта единственным соответствующим ей ключом или разбита.

Вася положил ключи в некоторые из копилков (он помнит, какой ключ лежит в какой из копилков). Теперь Вася собрался купить машину, а для этого ему нужно достать деньги из всех копилков. При этом он хочет разбить как можно меньшее количество копилков (ведь ему еще нужно копить деньги на квартиру, дачу, вертолет...). Помогите Васе определить, какое минимальное количество копилков нужно разбить.

Формат ввода

В первой строке содержится число n — количество свинок-копилков ($1 \leq n \leq 100$). Далее идет n строк с описанием того, где лежит ключ от какой копилки: в i -й из этих строк записан номер копилки, в которой находится ключ от i -й копилки.

Формат вывода

Выведите единственное число: минимальное количество копилков, которые необходимо разбить.

Пример

Ввод

4
2
1
2
4

Вывод

2

Пояснение к алгоритму

Каждое число задает переход из одной вершины в другую, поэтому получается функциональный граф. Нужно посчитать количество циклов в таком графе. Для этого запускается обход из каждой еще не посещенной вершины. В массиве посещений хранится номер старта текущего обхода. Если обход пришел в вершину, которая уже была посещена в этом же запуске, значит найден новый цикл.

Сложность по времени

$O(n)$.

Сложность по памяти

$O(n)$.

Код

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n;
5  vector<int> a;
6  vector<int> v;
7
8  int main() {
9      cin >> n;
10
11     a.resize(n + 1);
12     for (int i = 1; i <= n; i++)
13         cin >> a[i];
14
15     v.assign(n + 1, 0);
16
17     int c = 0;
18
19     for (int i = 1; i <= n; i++) {
20         if (v[i])
21             continue;
22
23         int x = i;
24
25         while (!v[x]) {
26             v[x] = i;
27             x = a[x];
28         }
29
30         if (v[x] == i)
31             c++;
32     }
33
34     cout << c;
35
36     return 0;
37 }
```

3 Задача О

Текст задания

О. Долой списывание!

✓ Полное решение

Ограничение времени	2 секунды
Ограничение памяти	64 Мб
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Во время теста Михаил Дмитриевич заметил, что некоторые лкшата обмениваются записками. Сначала он хотел поставить им всем двойки, но в тот день Михаил Дмитриевич был добрым, а потому решил разделить лкшат на две группы: списывающих и дающих списывать, и поставить двойки только первым.

У Михаила Дмитриевича записаны все пары лкшат, обменявшихся записками. Требуется определить, сможет ли он разделить лкшат на две группы так, чтобы любой обмен записками осуществлялся от лкшонка одной группы лкшонку другой группы.

Формат ввода

В первой строке находятся два числа N и M — количество лкшат и количество пар лкшат, обменивающихся записками ($1 \leq N \leq 100, 0 \leq M \leq \frac{N(N-1)}{2}$). Далее в M строках расположены описания пар лкшат: два различных числа, соответствующие номерам лкшат, обменивающихся записками (нумерация лкшат идёт с 1). Каждая пара лкшат перечислена не более одного раза.

Формат вывода

Необходимо вывести ответ на задачу Павла Олеговича. Если возможно разделить лкшат на две группы, выведите «YES»; иначе выведите «NO».

Пример

Ввод 	Вывод 
3 2 1 2 2 3	YES

Пояснение к алгоритму

Граф проверяется на двудольность. Для этого каждой вершине пытаемся назначить один из двух цветов. Обход в ширину раскрашивает соседей в противоположный цвет. Если во время обхода находится ребро между вершинами одного цвета, то граф не является двудольным. Так как граф может быть несвязным, обход запускается из каждой еще не раскрашенной вершины.

Сложность по времени

$O(n + m)$, где n - количество вершин, а m - количество ребер.

Сложность по памяти

$O(n + m)$.

Код

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n, m;
6      cin >> n >> m;
7
8      vector<int> g[105];
9      int a, b;
10
11     for (int i = 0; i < m; i++) {
12         cin >> a >> b;
13         g[a].push_back(b);
14         g[b].push_back(a);
15     }
16
17     int c[105];
18     memset(c, -1, sizeof(c));
19
20     queue<int> q;
21
22     for (int i = 1; i <= n; i++) {
23         if (c[i] != -1)
24             continue;
25
26         c[i] = 0;
27         q.push(i);
28
29         while (!q.empty()) {
30             int u = q.front();
31             q.pop();
32
33             for (int v : g[u]) {
34                 if (c[v] == -1) {
35                     c[v] = c[u] ^ 1;
36                     q.push(v);
37                 } else if (c[v] == c[u]) {
38                     cout << "NO";
39                     return 0;
40                 }
41             }
42         }
43     }
44
45     cout << "YES";
46 }
```


4 Задача Р

Текст задания

Р. Авиаперелёты

✓ Полное решение

Ограничение времени	2 секунды
Ограничение памяти	256 Мб
Ввод	стандартный ввод или avia.in
Вывод	стандартный вывод или avia.out

Главного конструктора Петю попросили разработать новую модель самолёта для компании «Air Бубундия». Оказалось, что самая сложная часть заключается в подборе оптимального размера топливного бака.

Главный картограф «Air Бубундия» Вася составил подробную карту Бубундии. На этой карте он отметил расход топлива для перелёта между каждой парой городов.

Петя хочет сделать размер бака минимально возможным, для которого самолёт сможет долететь от любого города в любой другой (возможно, с дозаправками в пути).

Формат ввода

Первая строка входного файла содержит натуральное число n ($1 \leq n \leq 1\,000$) — число городов в Бубундии. Далее идут n строк по n чисел каждая. j -е число в i -й строке равно расходу топлива при перелёте из i -го города в j -й. Все числа не меньше нуля и меньше 10^9 . Гарантируется, что для любого i в i -й строчке i -е число равно нулю.

Формат вывода

Первая строка выходного файла должна содержать одно число — оптимальный размер бака.

Пример

Ввод 	Вывод 
4 0 10 12 16 11 0 8 9 10 13 0 22 13 10 17 0	10

Пояснение к алгоритму

Нужно найти минимальное значение ограничения, при котором граф становится сильно связным. Для фиксированного значения оставляются только ребра с весом не больше этого значения. Затем проверяется достижимость всех вершин из первой вершины и достижимость первой вершины из всех остальных с помощью обхода в обратном графе. Если обе проверки успешны, граф сильно связан при этом ограничении. Ответ ищется бинарным поиском по возможному значению ограничения.

Сложность по времени

$O(n^2 \log C)$, где n - количество вершин, а C - диапазон значений весов.

Сложность по памяти

$O(n^2)$.

Код

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n;
5  long long a[1001][1001];
6  int u[1001];
7
8  void dfs(int v, long long x) {
9      u[v] = 1;
10     for (int i = 1; i <= n; i++)
11         if (!u[i] && a[v][i] <= x)
12             dfs(i, x);
13 }
14
15 void rdfs(int v, long long x) {
16     u[v] = 1;
17     for (int i = 1; i <= n; i++)
18         if (!u[i] && a[i][v] <= x)
19             rdfs(i, x);
20 }
21
22 bool ok(long long x) {
23     memset(u, 0, sizeof(u));
24     dfs(1, x);
25     for (int i = 1; i <= n; i++)
26         if (!u[i])
27             return false;
28
29     memset(u, 0, sizeof(u));
30     rdfs(1, x);
31     for (int i = 1; i <= n; i++)
32         if (!u[i])
33             return false;
34
35     return true;
36 }
37
38 int main() {
39     cin >> n;
40
41     for (int i = 1; i <= n; i++)
42         for (int j = 1; j <= n; j++)
43             cin >> a[i][j];
44
45     long long l = 0, r = 1e9;
46
47     while (l < r) {
48         long long m = (l + r) / 2;
49         if (ok(m))
50             r = m;
51         else
52             l = m + 1;
53     }
54
55     cout << l;
56 }
```